

EXP14: Alpha-Beta Pruning Algorithm for Gaming

Name: Y. Roshitha (192424400)

Aim:

To develop a Python program that implements the Alpha-Beta Pruning Algorithm to optimize the Minimax search by eliminating branches that cannot influence the final decision.

Algorithm:

Step 1: Represent the game tree using the leaf node values, and initialize $\alpha = -\infty$ and $\beta = +\infty$.

Step 2: Start the recursive alpha-beta function from the root node with the Maximizer's turn.

Step 3: If the current node is a leaf node, return its value.

Step 4: If it is the Maximizer's turn, update the best value and α after evaluating each child.

Step 5: If it is the Minimizer's turn, update the best value and β after evaluating each child.

Step 6: If β becomes less than or equal to α at any point, prune the remaining branches at that node.

Step 7: Repeat Steps 3 to 6 for every node until the root node's value is computed.

Step 8: Display the optimal value for the Maximizer and the total number of nodes visited after pruning.

Source Code:

```
# Program to implement Alpha-Beta Pruning Algorithm for Gaming

import math

nodes_visited = 0

def alpha_beta(depth, node_index, is_max, scores, max_depth, alpha, beta):
    global nodes_visited
    nodes_visited += 1

    if depth == max_depth:
        return scores[node_index]

    if is_max:
        best = -math.inf
        for i in range(2):
            value = alpha_beta(depth + 1, node_index * 2 + i, False, scores, max_depth, alpha, beta)
            best = max(best, value)
            alpha = max(alpha, best)
            if beta <= alpha:
                break
        return best
```

```

else:
    best = math.inf
    for i in range(2):
        value = alpha_beta(depth + 1, node_index * 2 + i, True, scores, max_depth, alpha,
beta)
        best = min(best, value)
        beta = min(beta, best)
        if beta <= alpha:
            break
    return best

```

```

scores = [3, 5, 2, 9, 12, 5, 23, 23]
max_depth = int(math.log2(len(scores)))

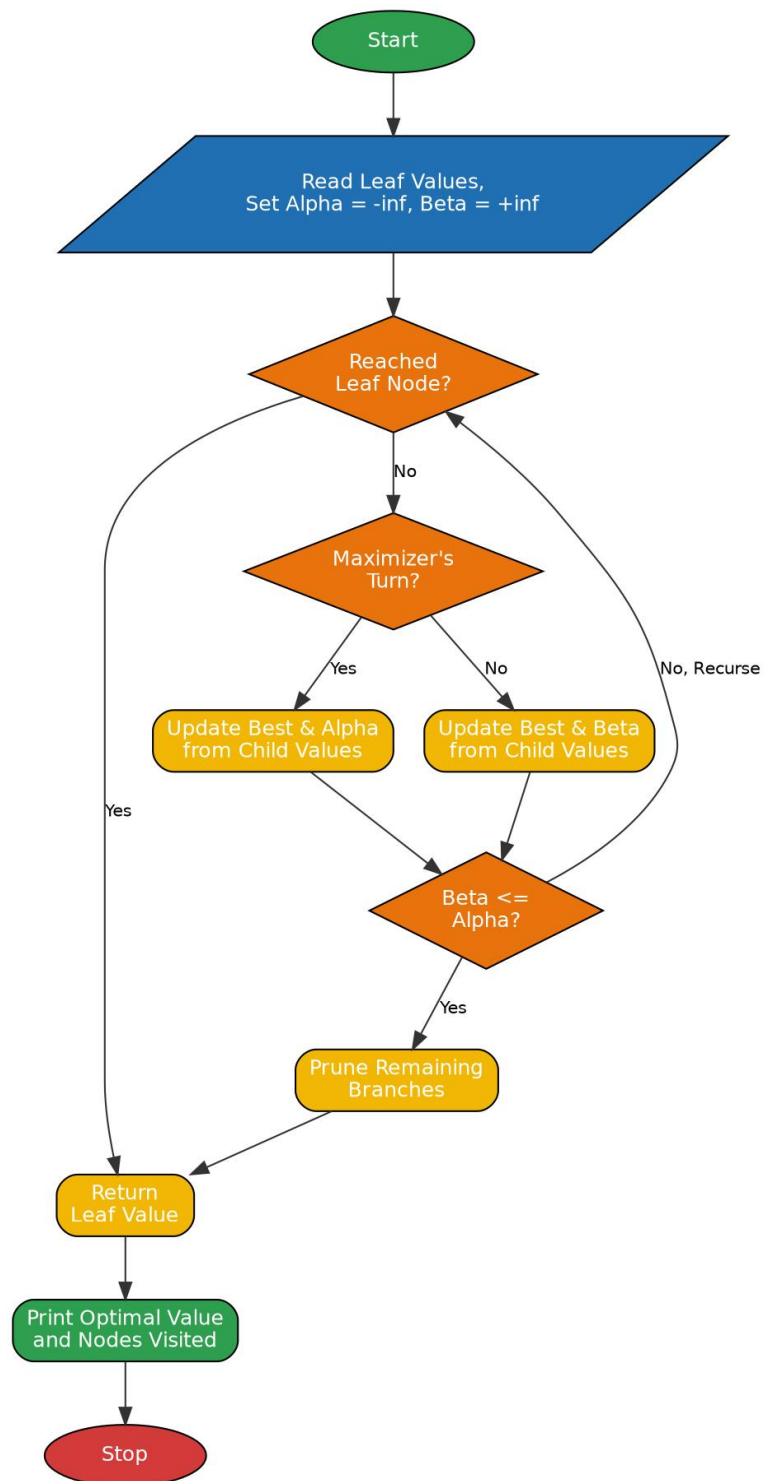
```

```

optimal_value = alpha_beta(0, 0, True, scores, max_depth, -math.inf, math.inf)
print("Leaf node values :", scores)
print("Tree depth      :", max_depth)
print("Optimal value for the Maximizer:", optimal_value)
print("Total nodes visited (with pruning):", nodes_visited)

```

Flowchart:



Output:

```
>>> ===== RESTART: C:/Users/yanar/Documents/CSA1717/EXP14.py =====  
Leaf node values : [3, 5, 2, 9, 12, 5, 23, 23]  
Tree depth      : 3  
Optimal value for the Maximizer: 12  
Total nodes visited (with pruning): 14  
>>> |
```

Result:

Thus the program to implement the Alpha-Beta Pruning Algorithm for gaming was written, executed successfully and the output was verified.